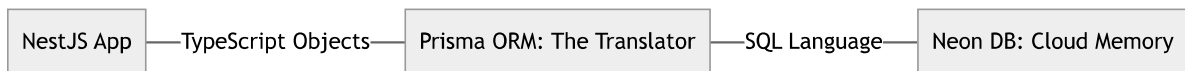


Day 2: Database Architecture & Prisma (The Ultimate Guide)

This guide explains how to connect your app to a Cloud Database and design your data models.

The Data Flow Diagram



Step 1: Secure Configuration (.env)

File: `.env`

We store your database “address” (Connection String) in a safe, hidden file.

```
DATABASE_URL="postgresql://user:pass@hostname/neondb?sslmode=require"
```

Deep Explorer:

This is called **Environment Separation**. By keeping secrets in a `.env` file, we ensure they aren’t accidentally shared when the code is uploaded to GitHub.

Step 2: The Data Blueprint (schema.prisma)

File: `prisma/schema.prisma`

We define exactly how our data should look.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id        Int        @id @default(autoincrement())
  email     String      @unique
  password  String
  role      Role        @default(USER)
  bookings Booking[]
}
```

```

}

model Car {
  id          Int          @id @default(autoincrement())
  brand       String
  model       String
  pricePerDay Float
  bookings    Booking[]
}

model Booking {
  id          Int          @id @default(autoincrement())
  startDate   DateTime
  endDate     DateTime
  userId      Int
  user        User        @relation(fields: [userId], references: [id])
  carId       Int
  car         Car         @relation(fields: [carId], references: [id])
}

enum Role {
  USER
  ADMIN
}

```

💡 Deep Explainer:

- **Relational Data:** Notice how `Booking` links to both `User` and `Car`. Prisma handles these “Foreign Key” connections for us automatically.

🔧 Step 3: Synchronization (Push & Generate)

Run these commands to apply your design to the cloud and prepare your code.

```

# 1. Update the Cloud Database tables
npx prisma db push

# 2. Generate the TypeScript "Prisma Client"
npx prisma generate

```

🔧 Step 4: The Database Bridge (Prisma Service 🐉)

File: `src/prisma/prisma.service.ts`

The service that keeps the connection alive.

```

import { Injectable, OnModuleInit } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

```

```
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit {
  .. async onModuleInit() {
    .... // Lifecycle Hook: Connect to DB as soon as the app starts
    .... await this.$connect();
    .. }
  }
}
```

⚠️ Key Learning: Version Compatibility

- **Issue:** Prisma 7 requires Node 22+.
 - **Solution:** We used **Prisma 6** to match your **Node 20** environment.
 - **Lesson:** A stable environment is more important than using the absolute newest version.
-

✅ Day 2 Graduation

Visit: `http://localhost:3000/db-test`

If it shows `userCount: 0`, you have successfully mastered database integration! ☁️ 🌐 🏆